

CryptoStream

Market Data Engineering Platform

End-to-End Technical Project Report

CoinGecko API -> External Ingestion -> Bronze -> Silver -> Gold

Prepared for technical review, portfolio presentation, and job interviews

Author: Swapnil Take | Version: 1.0 | Report created on 2023-08-10

Repository

github.com/swapniltake1/CryptoStream-Real-Time-Market-Data-Engineering-Platform

This report is based on the repository implementation and its README/documentation. Where the written design and current code differ, the report explicitly identifies the implementation status instead of presenting planned capabilities as completed features.

Table of Contents

1. Executive Summary
2. Business Problem and Objectives
3. Solution Architecture
4. Technology Stack
5. Source API and External Ingestion
6. Bronze Layer
7. Silver Layer: Cleansing and Data Quality
8. Gold Layer: Analytics
9. Incremental Processing and Idempotency
10. Pipeline Orchestration
11. Monitoring and Observability
12. Data Model and Data Dictionary
13. Repository Structure and Implementation Map
14. End-to-End Execution Runbook
15. Technical Design Review: Strengths and Gaps
16. Interview Explanation and Key Questions
17. Future Enhancements
18. Conclusion

Appendix A. Representative Code Patterns

Primary implementation references include the project README, external ingestion script, Bronze ingestion notebook, Silver transformation, Gold analytics, pipeline orchestrator, monitoring notebook, and file-tracking design document.

1. Executive Summary

CryptoStream is an enterprise-style cryptocurrency market-data engineering platform that turns CoinGecko market responses into governed, analytics-ready datasets using a Databricks Lakehouse and Medallion Architecture. The repository describes the platform as metadata-driven, scalable, maintainable and focused on data quality and operational observability. ■cite not needed in generated file■

The implemented flow is intentionally layered: a Python external-ingestion component calls the CoinGecko markets endpoint, attaches batch-level metadata, and persists JSON payloads. A Databricks Bronze process discovers new JSON files, tracks processed files in a Delta control table, extracts source records and writes them to a Bronze Delta table. Silver applies validation, deduplication, standardization and a derived volatility metric. Gold creates a latest market snapshot, top gainers, top losers and aggregate market metrics. A pipeline orchestrator runs the stages sequentially and a monitoring notebook checks freshness, record counts and data-quality indicators.

The project demonstrates the core competencies expected from a modern data engineer: API ingestion, Python, PySpark, Delta Lake, Medallion Architecture, schema handling, data-quality rules, incremental processing, file-level idempotency, orchestration, observability, and analytics modeling.

Architecture at a glance

Stage	Input	Processing	Output
Source	CoinGecko REST API	Market-data request	JSON response
External ingestion	API response	Batch metadata + persistence	Timestamped JSON
Bronze	JSON files	Parse + metadata + control-table tracking	main.crypto_bronze.coingecko_market_data
Silver	Bronze Delta	Validate + deduplicate + standardize + derive metrics	main.crypto_silver.crypto_market
Gold	Silver Delta	Snapshot + movers + aggregations	main.crypto_gold.*
Monitoring	Bronze/Silver/Gold	Freshness + counts + quality checks	Operational status

The repository README positions real-time streaming, Auto Loader, Unity Catalog, ML anomaly detection, enhanced lineage and CI/CD as future enhancements. The current implementation is therefore best described as a batch micro-batch style market-data platform with a path toward real-time streaming, rather than a fully streaming production platform today.

2. Business Problem and Objectives

Cryptocurrency markets change continuously. A practical analytics platform must ingest changing market values reliably, retain the original source information for traceability, prevent duplicate processing, validate records before analytics consumption, and expose concise business views such as latest price, market capitalization, trading volume, gainers and losers.

Core problem: a raw API response is not automatically a trustworthy analytical dataset. It needs operational metadata, repeatable processing, data-quality controls, historical traceability and a controlled path from raw to curated to business-ready data.

Primary objectives

- Ingest selected cryptocurrency market data from CoinGecko using Python.
- Preserve source payloads and attach batch, source and ingestion metadata.
- Use Delta Lake tables to create durable Bronze, Silver and Gold layers.
- Apply validation rules for required fields and non-negative monetary metrics.
- Remove duplicate observations using a deterministic partitioning strategy.
- Create a file-level control mechanism so successful files are not reprocessed.
- Produce business-ready analytics: market snapshot, top gainers, top losers and market metrics.
- Provide pipeline orchestration and operational monitoring.

Tracked assets in the current external-ingestion script are Bitcoin, Ethereum, Solana, Cardano, Ripple and Dogecoin, with USD as the target currency. ■cite not needed in generated file■

3. Solution Architecture

The architecture follows a classic Lakehouse Medallion pattern. Raw data is separated from curated data, while Gold tables are designed around analytical use cases. This separation supports traceability, quality controls and independent evolution of ingestion and business logic.

CoinGecko API REST market endpoint	Python Ingestion requests + metadata + batch ID	JSON Landing timestamped files
Bronze Raw Delta + source metadata	Control Table processed_files_control	Auditability file/batch traceability
Silver validation + dedup + standardization	Quality VALID / rejected logic	Business Transform 24h volatility
Gold Snapshot latest observation per coin	Top Movers gainers + losers	Market Metrics market cap, volume, averages
Databricks SQL / Dashboards analytics consumption	Monitoring freshness + counts + quality	Orchestrator ordered execution

Data flow: CoinGecko -> Python external ingestion -> timestamped JSON -> Bronze Delta -> Silver Delta -> Gold Delta -> analytics/monitoring.

Separation of concerns: ingestion owns acquisition and source metadata; Bronze owns raw persistence and file tracking; Silver owns quality and canonicalization; Gold owns analytical semantics; monitoring owns operational health checks; orchestration owns execution order.

4. Technology Stack

Technology	Role in project	Why it matters
Python 3.x	External API ingestion and pipeline utilities	Simple, portable API integration and operational scripting
requests	CoinGecko HTTP client	HTTP status/error handling and JSON retrieval
PySpark	Distributed transformations	Scalable DataFrame-based processing
Databricks	Lakehouse execution environment	Managed Spark, notebooks and orchestration
Delta Lake	Bronze/Silver/Gold persistence	Transactional table format and MERGE/upsert support
SQL	Analytical and table-oriented access	Business-friendly querying and downstream consumption
GitHub	Version control and project collaboration	Source history, reproducibility and portfolio presentation
JSON	External landing format	Natural representation of REST API payloads

The repository README explicitly describes Python, PySpark, Databricks Lakehouse, Delta Lake and CoinGecko as core technologies. █cite not needed in generated file█

Architectural choice: JSON is retained at the landing boundary because it preserves the external payload shape. Spark then infers/reads the records and the Silver layer establishes a more controlled analytical schema.

5. Source API and External Ingestion

The external ingestion script defines the CoinGecko markets endpoint, USD as the target currency, and a configured list of six coins. It builds query parameters for currency, IDs, market-cap ordering, page size, page number and sparkline behavior. The script calls the endpoint with a 30-second timeout and raises an exception for HTTP errors.

Batch metadata design

Metadata field	Purpose
project_name	Identifies the data product/pipeline
source_system	Identifies CoinGecko as the source
source_endpoint	Captures the exact API endpoint used
target_currency	Records the valuation currency
ingestion_timestamp	Records when the payload was acquired
batch_id	Provides a unique processing identifier
record_count	Captures number of source records in the payload

The script creates a batch ID using an UTC timestamp plus an 8-character UUID fragment. It then wraps the API data inside a metadata/data JSON envelope and writes a timestamped filename to both the external-ingestion data directory and project-level data directory. ■cite not needed in generated file■

The current source script has explicit exception handling for timeout, connection errors, HTTP errors and unexpected exceptions. The README also describes retry/rate-limit handling as a platform capability, but the current external-ingestion implementation shown in the repository does not yet implement retry with exponential backoff. This distinction is important for a production-readiness review.

Representative ingestion pattern

```
params = {
    "vs_currency": "usd",
    "ids": ",".join(COINS),
    "order": "market_cap_desc",
    "per_page": 100,
    "page": 1,
    "sparkline": "false"
}

response = requests.get(
    API_URL,
    params=params,
    timeout=30
)
response.raise_for_status()
data = response.json()
```

6. Bronze Layer

The Bronze notebook consumes JSON generated by the external ingestion process. Its operational objective is to preserve the incoming market records while adding lineage and processing metadata needed for audit and downstream processing.

Bronze processing sequence

- List JSON files in the external-ingestion data directory.
- Create the processed-files control table if it does not already exist.
- Read successful filenames from the control table.
- Filter out files that have already completed successfully.
- Sort new files by modification time and process the oldest first.
- Read the JSON payload and extract metadata and data records.
- Convert the data array into a Spark DataFrame.
- Add Bronze metadata such as batch ID, source system, source endpoint and processing timestamps.
- Append records into `main.crypto_bronze.coingecko_market_data`.
- Write a SUCCESS record into `main.crypto_bronze.processed_files_control`.
- Verify record counts and recent batches.

Bronze target table: `main.crypto_bronze.coingecko_market_data`. The control table is `main.crypto_bronze.processed_files_control`. The control-table documentation specifies filename, path, size, modification time, processing time, batch ID, record count and status fields. ■cite not needed in generated file■

Important implementation note: the current Bronze notebook references `latest_file.name` while adding the `source_file` column, even though the selected variable earlier in the notebook is `file_to_process`. In a clean execution, this should be corrected to `file_to_process.name`. This is a concrete code-quality issue to address before calling the notebook production-ready.

7. Silver Layer: Cleansing and Data Quality

Silver is the canonical analytical layer. It reads Bronze data and applies business and technical quality rules before Gold consumption.

Validation rules implemented

- id must be non-null.
- symbol must be non-null.
- name must be non-null.
- current_price must be non-null and ≥ 0 .
- market_cap must be non-null and ≥ 0 .

The implementation computes rejected records as the difference between the Bronze count and the validated count. It then removes duplicates using a Spark window partitioned by coin ID and ingestion date, retaining the latest record by ingestion timestamp.

The canonical Silver schema renames **id** -> **coin_id** and **name** -> **coin_name**, converts last_updated into **market_timestamp**, carries source/ingestion metadata forward, and calculates a derived metric:

price_volatility_24h = (high_24h - low_24h) / low_24h * 100.

Silver target table: main.crypto_silver.crypto_market. The repository uses Delta MERGE logic keyed by coin_id and ingestion_date so repeated processing of the same daily observation updates rather than blindly inserting the same daily key.

Design consideration: the README recommends a business key such as coin_id + market_timestamp. The current Silver implementation uses coin_id + ingestion_date. For higher-frequency market data, market_timestamp is generally a more precise grain and should be evaluated if multiple observations per coin per day are expected.

8. Gold Layer: Analytics

Gold converts the Silver dataset into business-oriented views. The current implementation creates four analytical outputs.

Gold dataset	Business purpose	Main logic
market_snapshot	Latest state of each tracked coin	Latest market_timestamp per coin
top_gainers	Largest positive 24h movers	Order by 24h percentage change descending; top 10
top_losers	Largest negative 24h movers	Order by 24h percentage change ascending; top 10
market_metrics	Overall market-level summary	SUM, AVG, MIN, MAX and COUNT aggregations

The market snapshot includes current price, market cap, market-cap rank, total volume, 24h price change, 24h percentage change, calculated volatility, market timestamp and snapshot timestamp.

Market metrics calculate total market capitalization, total 24h volume, number of tracked coins, average 24h price change, maximum/minimum 24h price change and average 24h volatility.

The repository also contains separate transformation files under the generated pipeline directory for market metrics, market snapshot, top gainers and top losers, in addition to the main `gold_market_analytics.py` implementation.

Gold write strategy

- `market_snapshot`: overwrite the current snapshot.
- `top_gainers`: overwrite the current top-10 result.
- `top_losers`: overwrite the current top-10 result.
- `market_metrics`: append metrics records so historical metric observations can accumulate.

9. Incremental Processing and Idempotency

Idempotency means a pipeline can be safely re-run without producing unintended duplicate business data. CryptoStream approaches this at two levels: file-level tracking in Bronze and MERGE-based record handling in Silver.

File-level exactly-once pattern

Step	Behavior
1. Discover	List JSON files in the landing directory.
2. Inspect control table	Read files already marked SUCCESS.
3. Filter	Keep only filenames not present as successful.
4. Order	Sort new files by modification time.
5. Process	Load the oldest unprocessed file.
6. Commit status	After successful Bronze write, append SUCCESS to control table.
7. Retry behavior	A failed file is not marked SUCCESS, so a later run can retry it.

The file-tracking document explicitly states the objectives: prevent duplicate records, preserve an audit trail, process chronologically, and allow failed files to be retried without reprocessing successful files.

The current design is strong for a file-based landing zone. For a true high-frequency streaming platform, this mechanism would normally evolve toward Auto Loader checkpoints, Structured Streaming checkpoints, event-time watermarks, deterministic event keys and/or Delta Change Data Feed depending on requirements.

Recommended production-grade business key

coin_id + market_timestamp is the preferred conceptual grain when every market observation must be preserved. It makes the dataset naturally time-series oriented and avoids collapsing multiple intraday observations into a single daily record.

10. Pipeline Orchestration

The pipeline orchestrator coordinates the end-to-end execution order. It invokes four notebooks with configured timeouts using `dbutils.notebook.run`.

Sequence	Notebook	Purpose	Timeout
1	Bronze Ingestion	Load new JSON files into Bronze	600 sec
2	Silver Transformation	Validate, deduplicate and standardize	600 sec
3	Gold Analytics	Create analytical datasets	600 sec
4	Pipeline Monitoring	Check freshness, counts and quality	300 sec

For each notebook, the orchestrator records start time, duration, SUCCESS/FAILED status and result or error message. If a stage fails, execution stops and the failure is surfaced in the final summary. This creates a simple dependency-aware control plane.

The orchestration pattern is appropriate for a portfolio-scale Databricks implementation. In a larger enterprise, the same dependency graph could be managed with Databricks Workflows, Airflow, Azure Data Factory, AWS Step Functions or another enterprise scheduler, while retaining the same logical stage boundaries.

Operational principle: fail fast at the orchestration boundary, but preserve enough metadata to diagnose the failing stage.

11. Monitoring and Observability

The monitoring notebook is designed around three operational dimensions: freshness, volume and quality.

Monitoring area	Current check	Interpretation
Freshness	Time since max ingestion_timestamp	FRESH < 1h; STALE < 24h; CRITICAL >= 24h
Layer volume	Counts in Bronze, Silver and Gold snapshot	Detect unexpected drops or missing layers
Data quality	Null prices, null market caps, negative prices	Detect malformed or incomplete records
Market behavior	Average 24h change and max volatility	Basic analytical health indicators

The repository README additionally describes desired operational metadata such as pipeline name, batch ID, start/end timestamps, records read/processed/inserted/updated/rejected, pipeline status and error message.

A production observability implementation should persist these metrics into an audit table rather than relying only on notebook print output. That would enable historical SLA reporting, alerting and trend analysis.

Suggested operational KPIs

- Data freshness lag.
- Pipeline duration by stage.
- Records received vs. records accepted/rejected.
- Duplicate rate.
- API request success/failure rate.
- Time since last successful run.
- Silver-to-Gold record reconciliation.
- Schema-drift events.

12. Data Model and Data Dictionary

The source payload contains market attributes including identity, price, market capitalization, volume, 24h high/low, price changes, supply information, all-time-high/low values and a last-updated timestamp. The Silver layer deliberately selects the fields most relevant to market analytics.

Field	Silver meaning	Typical type	Business meaning
coin_id	Stable source identifier	STRING	Unique cryptocurrency identifier
symbol	Trading symbol	STRING	Short market symbol
coin_name	Human-readable name	STRING	Display name
current_price	Current market price	DOUBLE	Latest quoted USD price
market_cap	Market capitalization	DOUBLE/LONG	Approximate total market value
market_cap_rank	Market ranking	INT	Relative market-cap rank
total_volume	24h trading volume	DOUBLE/LONG	Trading activity indicator
high_24h	24h high	DOUBLE	Highest observed price in 24h window
low_24h	24h low	DOUBLE	Lowest observed price in 24h window
price_change_24h	Absolute price movement	DOUBLE	Price delta over 24h
price_change_percentage_24h	Percentage price movement	DOUBLE	Relative 24h change
market_timestamp	Source market timestamp	TIMESTAMP	Time represented by source observation
ingestion_timestamp	Pipeline ingestion time	TIMESTAMP	When the record entered the platform
price_volatility_24h	Derived volatility proxy	DOUBLE	24h high-low range as % of low
record_quality	Quality classification	STRING	Current implementation labels accepted records VALID

A key modeling principle is to distinguish **event time** (market_timestamp) from **processing time** (ingestion_timestamp). This separation is essential for time-series analytics, late-arriving data handling and accurate operational monitoring.

The repository's source JSON example contains six records and demonstrates the complete metadata envelope: project name, source system, endpoint, currency, ingestion timestamp, batch ID and record count, followed by the market-data array.

13. Repository Structure and Implementation Map

The current main branch contains the following major implementation areas. Empty documentation/SQL placeholders are present in the repository and should be treated as planned or partially scaffolded assets rather than completed modules.

Path	Role / status
README.md	Project overview, architecture, features, setup and future roadmap
external_ingestion/coingecko_ingestion.py	Python API ingestion and JSON landing
external_ingestion/data/	Generated JSON landing examples
bronze/bronze_from_external_ingestion.py	Databricks Bronze ingestion and file control
silver/silver_transformation.py	Silver validation, deduplication and canonicalization
gold/gold_market_analytics.py	Gold snapshot, movers and metrics
monitoring/pipeline_orchestrator.py	Sequential end-to-end notebook orchestration
monitoring/pipeline_monitoring.py	Freshness, layer-count and quality checks
monitoring/docs/file_tracking_mechanism.md	Detailed exactly-once file tracking design
cryptostream_market_data_pipeline_0c4d49a7/transformations/	Additional Bronze/Silver/Gold transformation assets
config/	Configuration scaffolding; some files are currently empty
sql/	SQL scaffolding; current table/analytics files are empty
docs/	Architecture/data-dictionary/pipeline-design placeholders
.github/workflows/	Repository workflow asset

The repository tree shows a deliberate separation between the runnable top-level Python/Databricks stages and a generated pipeline transformation directory. Consolidating duplicate transformation assets into one canonical implementation path would reduce maintenance overhead over time.

14. End-to-End Execution Runbook

Step 1 - Generate source data. Run the external Python ingestion script. It calls CoinGecko, generates a batch ID, builds metadata and writes a timestamped JSON file.

Step 2 - Land the file where Databricks can access it. The current Bronze notebook expects the JSON under the configured `external_ingestion/data` workspace path.

Step 3 - Run Bronze. Bronze discovers files, consults the control table, selects the oldest unprocessed file, parses the payload and appends records to the Bronze Delta table.

Step 4 - Run Silver. Silver reads Bronze, validates required fields and numeric ranges, deduplicates, standardizes column names/types and derives 24h volatility.

Step 5 - Run Gold. Gold creates the latest snapshot, top 10 gainers, top 10 losers and aggregate market metrics.

Step 6 - Run monitoring. Check freshness, record counts and Silver data-quality metrics.

Step 7 - Validate outputs. Reconcile source record count, Bronze batch count, Silver accepted/rejected counts and Gold snapshot count.

Operational validation checklist

Check	Expected outcome
API request	HTTP success and JSON response
Batch metadata	Unique batch ID and correct record count
Landing file	Timestamped JSON exists
Control table	File is marked SUCCESS only after Bronze write
Bronze	Records visible with source/batch metadata
Silver	Required fields valid; duplicates controlled
Gold	Snapshot and mover tables populated
Monitoring	Freshness and quality checks complete

For local setup, the README lists Python 3.8+, pip, PySpark/Spark-compatible execution and Databricks/Delta Lake as prerequisites. Credentials should be provided through environment/secret management and never committed to source control.

15. Technical Design Review: Strengths and Gaps

This section separates what is strong in the current implementation from items that should be addressed before positioning the project as a production-grade real-time platform.

Strengths

Strength	Why it is valuable
Medallion architecture	Clear separation between raw, curated and analytical concerns.
Batch metadata	Strong lineage foundation through batch ID, source and timestamps.
File control table	Prevents successful files from being repeatedly ingested.
Silver validation	Basic completeness and numeric validity checks protect Gold.
Delta MERGE	Introduces deterministic upsert behavior in Silver.
Business-oriented Gold	Produces useful outputs rather than only normalized data.
Orchestration	Defines explicit stage dependencies and failure behavior.
Monitoring	Introduces freshness, volume and quality observability.

Production-readiness gaps

Gap / observation	Recommended improvement
No retry/backoff in current API script	Implement bounded exponential backoff for 429/5xx/timeouts and respect API rate limits.
Bronze variable mismatch	Replace latest_file.name with file_to_process.name in source_file assignment.
Daily Silver key	Consider coin_id + market_timestamp for true time-series grain.
Quality classification	Persist INVALID/QUARANTINED records separately rather than only filtering them out.
Audit metrics	Persist run-level metrics to a Delta audit table for historical monitoring.
Gold overwrite semantics	Use deterministic snapshot semantics or partitioned history where historical Gold is required.
Schema evolution	Add explicit schema/version checks and alert on unexpected source changes.
Streaming claim	Use Structured Streaming/Auto Loader/WebSocket ingestion if true real-time behavior is required.
Testing	Add automated unit, integration and data-quality tests; current SQL/docs scaffolding is incomplete.
Security	Use secret scopes/environment variables and avoid hard-coded user-specific workspace paths.

16. Interview Explanation and Key Questions

A concise interview explanation can be structured around **problem -> architecture -> implementation -> quality -> operations -> scalability**.

90-second project explanation

“CryptoStream is a cryptocurrency market-data engineering platform I built using Python, PySpark, Databricks and Delta Lake. I ingest selected market data from CoinGecko into timestamped JSON files with batch metadata. In Databricks, I use a Medallion Architecture: Bronze preserves source data and tracks processed files through a Delta control table; Silver validates required fields, removes duplicates, standardizes the schema and calculates a 24-hour volatility metric; Gold creates the latest market snapshot, top gainers, top losers and market-level metrics. I also built an orchestrator to run Bronze, Silver, Gold and monitoring in sequence, plus monitoring for freshness, record counts and data quality. The next production step would be to move the ingestion boundary to Auto Loader or Structured Streaming and add stronger retry, audit and automated testing.”

High-value interview questions

Question	Strong answer direction
Why Medallion Architecture?	It separates raw preservation, quality/standardization and business analytics so each layer has a clear contract.
How do you prevent duplicate file ingestion?	A Delta control table stores successfully processed filenames; Bronze filters those files out before processing.
How is Silver different from Bronze?	Bronze preserves source fidelity plus ingestion metadata; Silver establishes a trusted analytical schema and quality rules.
Why use MERGE?	MERGE supports deterministic upsert behavior for an identified business grain and makes repeat processing safer.
What is event time vs ingestion time?	Market timestamp describes the source observation; ingestion timestamp describes when the platform processed it.
How would you make it real-time?	Use WebSocket/API streaming where supported, Structured Streaming or Auto Loader, checkpoints, watermarking and event-time-aware logic.
How would you handle API rate limits?	Bounded retries with exponential backoff, 429-aware handling, request pacing and centralized observability.
What would you monitor?	Freshness, stage duration, input/output counts, rejects, duplicates, API failures, schema drift and last-success timestamp.
How would you test it?	Unit tests for transformations, integration tests across Bronze/Silver/Gold, and data-quality tests for schema/nulls/duplicates/ranges.

17. Future Enhancements

The repository README already identifies several future directions. The following roadmap prioritizes them into an enterprise delivery sequence.

Priority	Enhancement	Outcome
P0	Fix Bronze source_file variable and harden API retry/backoff	Improved reliability
P0	Move secrets/configuration to managed secret/config services	Security and portability
P0	Add automated tests and run them in CI	Regression protection
P1	Databricks Auto Loader / Structured Streaming	Near-real-time ingestion
P1	Unity Catalog governance	Centralized access control and lineage
P1	Run audit Delta table	Historical operational reporting
P1	Quarantine invalid records	Better data-quality remediation
P2	Schema-drift detection and alerting	Resilience to source changes
P2	ML anomaly detection	Automated unusual-market-event detection
P2	Dashboard layer	Business-facing market monitoring

A mature target architecture would support continuous ingestion, event-time processing, governed tables, automated data-quality gates, alerting and reproducible CI/CD deployment while preserving the same Bronze/Silver/Gold conceptual contracts.

18. Conclusion

CryptoStream is a strong portfolio-grade data engineering project because it demonstrates more than a simple API-to-DataFrame transformation. It addresses the operational lifecycle of data: acquisition, raw preservation, metadata, incremental processing, quality, canonicalization, analytics, orchestration and monitoring.

The implementation already contains the most important architectural building blocks of a modern Lakehouse pipeline. Its strongest differentiator is the explicit file-tracking mechanism combined with a Medallion design and business-focused Gold outputs.

The main opportunity is to close the gap between the documented target architecture and the current implementation: production-grade retries, true streaming, stronger audit persistence, automated testing, secure configuration, schema-drift handling and a more precise time-series business key.

For interviews, the project can be positioned as a practical demonstration of how a data engineer moves from an external API payload to trustworthy analytical datasets while balancing data quality, idempotency, operational reliability and scalability.

Final project positioning

A Databricks Lakehouse market-data platform that ingests CoinGecko data, preserves raw source payloads, applies Silver-layer quality and canonicalization, produces Gold analytics, tracks processed files for idempotency, orchestrates the complete workflow and exposes operational monitoring.

Repository: github.com/swapniltake1/CryptoStream-Real-Time-Market-Data-Engineering-Platform

Appendix A. Representative Code Patterns

The following snippets summarize the principal implementation patterns in the repository. They are intentionally shortened so the report explains the architecture rather than duplicating the complete source files.

A. Batch ID generation

```
timestamp = datetime.now(timezone.utc).strftime("%Y%m%d%H%M%S")
unique_id = str(uuid.uuid4())[:8]
batch_id = f"BATCH_{timestamp}_{unique_id}"
```

B. Silver validation

```
df_validated = df_bronze.filter(
    col("id").isNotNull() &
    col("symbol").isNotNull() &
    col("name").isNotNull() &
    col("current_price").isNotNull() &
    (col("current_price") >= 0) &
    col("market_cap").isNotNull() &
    (col("market_cap") >= 0)
)
```

C. Deduplication

```
window_spec = (
    Window.partitionBy("id", "ingestion_date")
        .orderBy(col("ingestion_timestamp").desc())
)

df_deduped = (
    df_validated
    .withColumn("row_num", row_number().over(window_spec))
    .filter(col("row_num") == 1)
    .drop("row_num")
)
```

D. Gold latest snapshot

```
window_spec = (
    Window.partitionBy("coin_id")
        .orderBy(col("market_timestamp").desc())
)

df_market_snapshot = (
    df_silver
    .withColumn("row_num", row_number().over(window_spec))
    .filter(col("row_num") == 1)
    .drop("row_num")
)
```

E. Orchestration concept

```
for notebook in NOTEBOOKS:
    result = dbutils.notebook.run(
        notebook["path"],
        timeout_seconds=notebook["timeout"]
    )
    # record SUCCESS / FAILED and duration
```

Source basis: CryptoStream repository main branch reviewed on 28 August 2026. Key source files include README.md, external_ingestion/coingecko_ingestion.py, bronze/bronze_from_external_ingestion.py, silver/silver_transformation.py, gold/gold_market_analytics.py, monitoring/pipeline_orchestrator.py, monitoring/pipeline_monitoring.py and monitoring/docs/file_tracking_mechanism.md.